Project – Low Level Design

on

Healthcare Content Generator

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number

S.no	Student name	Enrollment Number
1	MIMANSA KASHYAP	EN22IT301054
2	HARSH GUPTA	EN24CA5030057
3	ADITYA BAIRAGI	EN24CA5030009
4	SANDHYA PATIDAR	EN24CA5030150
5	SHUBHANJALI ROY	EN24CA5030161
6	APARNA SHARMA	EN24CA5030023

Group Name: Group 12D12

Project Number: GAI-22

Industry Mentor Name: Yash

University Mentor Name: Ajaj Khan

Academic Year: 2024-2026

Table of Contents

Page No.

1. Introduction

The Healthcare Content Generator is a specialized Generative AI application designed to assist healthcare professionals in creating accurate, well-structured, and professional medical documentation. In the healthcare sector, doctors, nurses, and medical staff often spend a significant amount of time preparing reports, patient summaries, clinical notes, and other documentation. This process can be time-consuming and may sometimes lead to inconsistencies in formatting or terminology. The proposed system uses advanced Generative AI techniques and prompt engineering to convert simple user inputs or medical topics into high-quality professional content. By integrating powerful language models through an LLM API, the system can generate structured and medically appropriate responses. The application also ensures that the generated content follows standard medical terminology and professional tone. As a result, the Healthcare Content Generator reduces manual effort, saves time, and improves the efficiency of healthcare professionals while maintaining clarity and consistency in medical documentation.

1.1 Scope of the document

This document provides a detailed low-level design of the Healthcare Content Generator system. It focuses on the internal architecture, data flow, component-level design, APIs, and database structure required for implementation. The purpose of this document is to serve as a technical blueprint for developers, ensuring clarity in how each module interacts within the system. It also defines the logic behind prompt generation, data retrieval from the vector database, and integration with the LLM API to produce accurate and professional healthcare content.

1.2 Intended Audience

This document is intended for software developers, system architects, project evaluators, and stakeholders who are involved in the design, development, and deployment of the system. Developers will use this document to understand how to implement different modules, while designers can refer to it for structuring the system efficiently. Additionally, it is useful for academic evaluation and project review purposes, as it clearly explains the working and structure of the application.

1.3. System Overview

The Healthcare Content Generator is designed as an intelligent AI-powered system that automatically generates healthcare-related content based on user input. The system works by accepting a simple prompt or topic from the user, such as a patient condition or medical procedure. This input is processed using carefully designed prompt engineering techniques that guide the AI model to generate structured and meaningful output. The system also integrates a vector database to store and retrieve relevant medical knowledge, guidelines, and reference content. When a user submits a request, the system retrieves relevant information from the vector database and combines it with the prompt before sending it to the language model through an API. The language model then generates detailed and professional healthcare content such as patient summaries, medical explanations, or documentation. The final output is displayed to the user in a structured and readable format. Overall, the system improves productivity and helps healthcare professionals create high-quality documentation quickly and efficiently.

2.System Design

2.1 Application Design

The application follows a layered architecture approach, consisting of a user interface layer, backend processing layer, and AI processing layer. The frontend allows users to input healthcare topics and select content templates. The backend, developed in Python, acts as the core processing unit, handling user requests, generating prompts, and coordinating with other modules. The AI layer includes the vector database and LLM API, which work together to fetch relevant context and generate accurate content. This modular design ensures flexibility, scalability, and ease of maintenance.

Architecture Type

- Layered Architecture (Frontend + Backend + AI Layer)

Modules

1. **User Interface**
    - Input topic
    - Select template
2. **Backend Server (Python**
    - Handles requests
    - Applies prompt engineering
3. **Prompt Engine**
    - Converts input → structured prompt
4. **Vector Database**
    - Stores embedding of medical knowledge
    - Retrieves relevant context
5. **LLM API Layer**
    - Generates final content
6. **Post-processing Module**
    - Formats output
    - Ensures medical tone

2.2 Process Flow

The process begins when a user enters a healthcare-related topic into the system. The backend receives this input and passes it to the prompt engineering module, which converts the raw input into a structured and context-rich prompt. This prompt is then enhanced by retrieving relevant medical information from

the vector database using similarity search. The enriched prompt is sent to the LLM API, which generates the required content based on the input and context. Finally, the generated output is processed and formatted to meet professional healthcare standards before being displayed to the user.

2.3 Information Flow

The information flow in the system is designed to ensure smooth and efficient data movement across different components. Initially, user input is sent to the backend server, where it is processed and transformed into a structured prompt. This prompt interacts with the vector database to

retrieve relevant contextual data, which is then combined and forwarded to the LLM API. The generated response is returned to the backend, where it undergoes formatting and validation before being delivered to the user. This structured flow ensures accuracy, consistency, and reliability of the generated content.

User Input → Backend → Prompt Engine → Vector DB
→ Context Retrieval → LLM API
→ Generated Content → Formatting → Output

2.4 Components Design

1. User Module

- Handles login and input
- Stores user history

2. Prompt Module

- Predefined templates:
  - Patient Summary
  - Diagnosis Report
- Dynamic prompt generation

3. Vector DB Module

- Stores embedding using:
  - Medical documents
- Performs similarity search

4. LLM Integration Module

- Sends request to API
- Receives generated text

5. Output Formatter

- Ensures:
  - Professional tone
  - Structured format
  - Medical terminology

2.5 Key Design Considerations

Several important factors are considered while designing the system. Accuracy is critical in healthcare applications, so the system uses reliable data sources and context retrieval methods. Consistency is maintained through predefined prompt templates, ensuring uniform output across different use cases. Scalability is achieved by designing modular components that can handle increasing user load. Security is also a major concern, as healthcare data must be protected from unauthorized access. Additionally, performance optimization is ensured by using efficient vector search techniques, and maintainability is supported through a clean and modular architecture.

1. Accuracy

- Use medical datasets in Vector DB

2. Consistency

- Prompt templates ensure uniform output

3. Scalability

- Microservice-ready design

4. Security

- Protect sensitive healthcare data

5. Performance

- Fast retrieval using Vector DB

6. Maintainability

- Modular architecture

2.6 API Catalogue

The system exposes a set of APIs to handle different functionalities. The "Generate Content" API allows users to submit a topic and receive generated healthcare content. The "Get Templates" API provides a list of available prompt templates that users can choose from. The "User History" API enables retrieval of previously generated content for a specific user. The "Store Content" API is used to save generated outputs into the database. These APIs are designed to be simple, scalable, and easy to integrate with the frontend and other services.

1. Generate Content API

```
POST /generate
```

Request:

```
{
  "topic": "Diabetes patient summary",
  "template": "patient_summary"
}
```

Response:

```
{
  "generated_text": "Patient is diagnosed with..."
}
```

2. Get Templates API

```
GET /templates
```

3. User History API

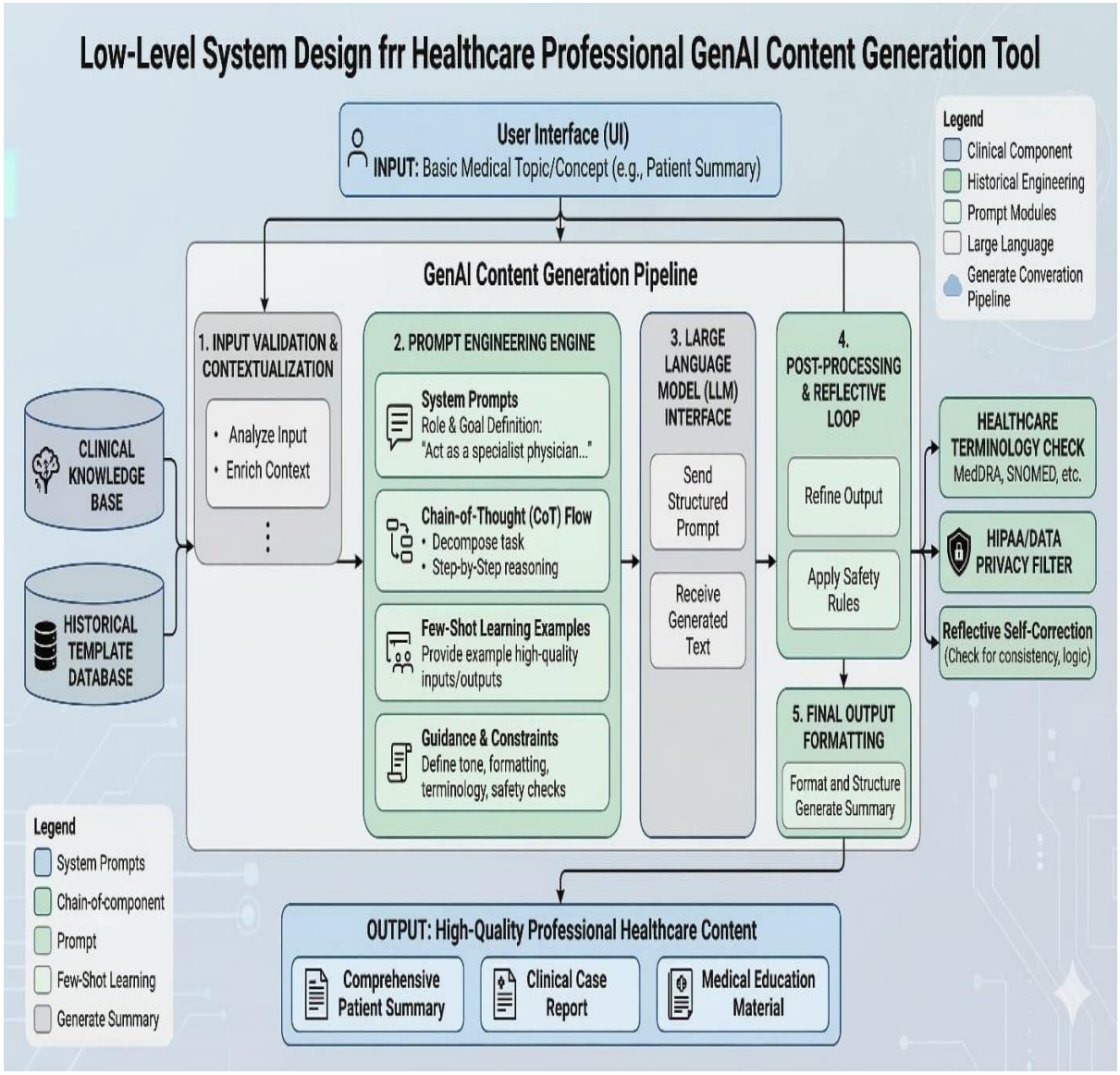
```
GET /history/{user_id}
```

4. Store Content API

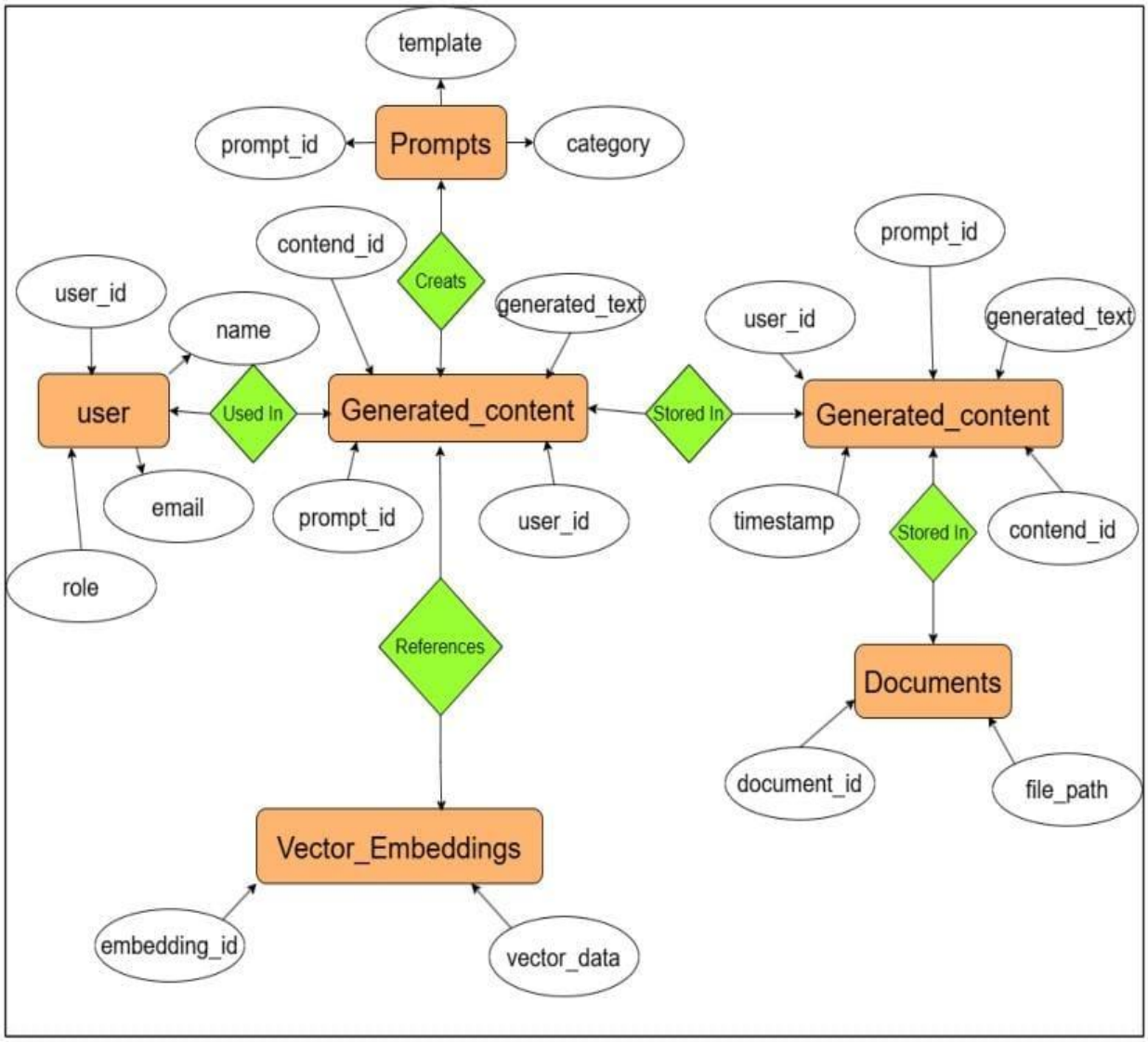
```
POST /store
```

3. System Architecture Diagram

The **Healthcare Content Generator** uses **Python, a Vector Database, and an LLM API** to generate healthcare content. First, the user enters a topic or request in the system. The Python backend processes the input and uses **prompt engineering** to create a structured prompt. Then, the system retrieves relevant information from the **vector database** to provide context. This prompt and context are sent to the **LLM API**, which generates professional healthcare content such as patient summaries. Finally, the generated output is formatted and displayed to the user.


Low-Level System Design frr Healthcare Professional GenAI Content Generation Tool
User Interface (UI)
INPUT: Basic Medical Topic/Concept (e.g., Patient Summary)
Legend
Clinical Component
Historical Engineering
Prompt Modules
Large Language
Generate Converation Pipeline
GenAI Content Generation Pipeline
1. INPUT VALIDATION & CONTEXTUALIZATION
• Analyze Input
• Enrich Context
2. PROMPT ENGINEERING ENGINE
System Prompts
Role & Goal Definition: "Act as a specialist physician..."
Chain-of-Thought (CoT) Flow
• Decompose task
• Step-by-Step reasoning
Few-Shot Learning Examples
Provide example high-quality inputs/outputs
Guidance & Constraints
Define tone, formatting, terminology, safety checks
3. LARGE LANGUAGE MODEL (LLM) INTERFACE
Send Structured Prompt
Receive Generated Text
4. POST-PROCESSING & REFLECTIVE LOOP
Refine Output
Apply Safety Rules
5. FINAL OUTPUT FORMATTING
Format and Structure Generate Summary
CLINICAL KNOWLEDGE BASE
HISTORICAL TEMPLATE DATABASE
HEALTHCARE TERMINOLOGY CHECK
MedDRA, SNOMED, etc.
HIPAA/DATA PRIVACY FILTER
Reflective Self-Correction
(Check for consistency, logic)
Legend
System Prompts
Chain-of-component
Prompt
Few-Shot Learning
Generate Summary
OUTPUT: High-Quality Professional Healthcare Content
Comprehensive Patient Summary
Clinical Case Report
Medical Education Material

4.ER Daigram


template
Prompts
prompt_id
category
contend_id
Creats
generated_text
user_id
name
user
Used In
Generated_content
Stored In
Generated_content
prompt_id
user_id
generated_text
email
prompt_id
user_id
timestamp
Stored In
contend_id
role
References
Documents
document_id
file_path
Vector_Embeddings
embedding_id
vector_data

References

1. **Brown, T., et al. (2020).** *Language Models are Few-Shot Learners.* Advances in Neural Information Processing Systems (NeurIPS). https://arxiv.org/abs/2005.14165
2. **OpenAI Documentation.** https://platform.openai.com/docs
3. **Lewis, P., et al. (2020).** *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* https://arxiv.org/abs/2005.11401
4. **Jurafsky, D., & Martin, J. (2023).** *Speech and Language Processing.* Stanford University.
5. **Vector Database Concepts – Pinecone Documentation.** https://www.pinecone.io/learn/vector-database/
6. **Python Official Documentation.** https://docs.python.org/3/
7. **Prompt Engineering Guide.** https://www.promptingguide.ai/
8. **Kimball, R. & Ross, M. (2013).** *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling.*
9. **Elmasri, R. & Navathe, S. (2016).** *Fundamentals of Database Systems.*
10. **IBM Cloud Education.** *What is Generative AI?* https://www.ibm.com/topics/generative-ai